

The Redemption — Infrastructure Design

Overview

The Redemption is a hotel loyalty point deduction service handling steady baseline traffic with sudden 10x spikes during flash sales. The infrastructure runs on AWS EKS in Singapore (ap-southeast-1) across three AZs. Zero downtime is a hard requirement. A failed redemption during a flash sale is direct revenue loss, not a latency blip.

A. Compute & Architecture

VPC: 10.0.0.0/16 across three AZs. Public subnets for the ALB and NAT gateways only. Private subnets for EKS workers. Database subnets (isolated, no route to the internet) reserved for ElastiCache when the application team is ready to deploy it.

NAT: One NAT gateway per AZ to prevent a single-AZ failure from killing egress for the whole cluster.

EKS: Managed control plane, workers in private subnets. Control plane logs (API, audit, authenticator, controllerManager, scheduler) shipped to CloudWatch. API server has public access enabled for initial deployment — restrict to a VPN/corporate CIDR via `eks_public_access_cidrs` before production go-live.

VPC Endpoints: - *Gateway endpoints* (no hourly charge): S3, DynamoDB — routes all S3 and DynamoDB traffic off the NAT gateway. - *Interface endpoints* (per-AZ hourly charge justified by call volume): ECR API, ECR DKR, STS — keeps container image pulls and IRSA token requests inside the VPC. - SQS, CloudWatch Logs, and Secrets Manager are not yet on VPC endpoints; those calls route through NAT gateways. Worth adding post-launch once traffic volume makes the cost trade-off clear.

Nodes: Baseline group of 3 on-demand nodes (one per AZ, always running) using `m6i.xlarge` or `m6a.xlarge` — AWS picks the cheapest available. Karpenter provisions burst capacity from spot pools when HPA creates pending pods. Spot is substantially cheaper than on-demand for the burst window, with on-demand as fallback.

B. Scalability Strategy

The 10x spike is the core architectural challenge. We tackle this in two layers:

Layer 1 — Pod scaling (HPA): We run 6 baseline replicas that scale up to 60 based on CPU (60% threshold) and memory (70% threshold). The scale-up is aggressive (0-second stabilization) to immediately absorb sudden spikes, while scale-down is conservative (5-minute cooldown) to prevent flapping as traffic normalizes.

Layer 2 — Node scaling (Karpenter): When the HPA creates pending pods, Karpenter provisions right-sized spot nodes in roughly 60 seconds. We've configured the NodePool with a mix of instance families (`m6i` , `m6a` , `m5` , `m5a` , `c6i` , `c6a`) in large, xlarge, and 2xlarge sizes, giving Karpenter enough options to reliably find spot capacity. Spot is substantially cheaper than on-demand for burst windows. Nodes expire after 720h (30 days) to force patch rotation automatically, and Karpenter consolidates underutilised nodes after 10 minutes to cut costs between spikes. Gateway VPC endpoints for S3 and DynamoDB keep high-volume data-plane traffic off the NAT gateways entirely.

C. Security & Networking

Network topology: ALB sits in public subnets with WAFv2 attached for edge protection (rate limiting 5 000 req/IP, AWS Managed Rules for OWASP Top 10 and known bad inputs). EKS workers are in private subnets with no direct internet access. AWS API calls for ECR, DynamoDB, S3, and STS go through VPC endpoints; SQS, CloudWatch Logs, and Secrets Manager calls route through NAT.

Pod-level IAM (IRSA): Each pod assumes its own IAM role via OIDC federation: - `redemption-service` — scoped to `redemption-*` DynamoDB tables, SQS queues, and Secrets Manager paths. - `fluent-bit` — CloudWatch Logs write access for `/eks/redemption-prod/*` log groups only. - `aws-load-balancer-controller` — EC2/ELB/WAFv2 permissions required for ALB provisioning. - `karpenter-controller` — EC2 fleet and pricing permissions for node provisioning.

We strictly avoid using broad node-level instance profiles. If a container is ever compromised, the blast radius is contained to that service's resources.

Network policies: Default deny on both ingress and egress at the namespace level. Explicit allows: - ALB public subnets → pods on port 8080 - Pods → CoreDNS in kube-system (UDP/TCP 53) - Pods → AWS services via VPC endpoints (TCP 443) - Pods → Redis subnet range on TCP 6379 (forward-compatibility placeholder for ElastiCache)

KMS: EKS secrets are encrypted at rest with a KMS key (auto-rotation enabled, 7-day deletion window). Application-level field encryption (e.g., PII in DynamoDB) would use a separate key — deferred because application code is out of scope.

TLS: ALB terminates TLS 1.3 (SSL policy `ELBSecurityPolicy-TLS13-1-2-2021-06`). Internal pod-to-pod communication is plaintext within the VPC — acceptable given the network policy controls.

D. Reliability & Observability

Handling an AZ failure: - `topologySpreadConstraints` with a `maxSkew: 1` ensures pods are evenly balanced across all available AZs. - A `PodDisruptionBudget` with `minAvailable: 4` guarantees that even if we lose an entire AZ (dropping from 6 replicas to 4), the application stays up. The PDB also prevents rolling updates or Karpenter consolidation from dropping our baseline below this safe threshold. - DynamoDB and ElastiCache (when deployed) are natively multi-AZ.

Handling a bad deployment: - `maxUnavailable: 0` ensures a rolling update never terminates an old pod before its replacement passes readiness checks. - A `preStop: sleep 15s` lifecycle hook gives the ALB 15 seconds to safely drain existing connections before the pod terminates. - We use three distinct probes: startup (generous, allowing for slow cold starts), readiness (to gate traffic), and liveness (strictly to restart genuinely hung pods). Collapsing these into a single probe is a common anti-pattern that can cause cascading restarts under heavy load.

Application data stores (IRSA policies and network egress pre-configured; table/queue provisioning is deferred to the application team): - *DynamoDB* (`redemption-transactions`) — primary transaction store. Recommended: PAY_PER_REQUEST billing with PITR enabled. - *SQS* (`redemption-events`) — async event processing. Recommended: 24h retention, 60s visibility timeout, KMS encryption. - *Secrets Manager* (`redemption/app-config`, provisioned by Terraform) — API keys and feature flags, auto-rotation enabled. - *ECR* (`redemption-service`, provisioned by Terraform) — container image registry, immutable tags, scan on push, KMS encrypted.

Monitoring: - Structured JSON logs shipped to CloudWatch Logs (90-day retention) via Fluent Bit. Fluent Bit runs as a DaemonSet in the `logging` namespace, deployed as part of the standard Kustomize overlay (`kubernetes/base/logging/`). Its IRSA role is provisioned by Terraform. - CloudWatch log groups: `/eks/redemption-prod/app/redemption` (application logs), `/aws/eks/redemption-prod/cluster` (control plane logs). - CloudWatch alarms (activated by setting `alb_deployed = true` in `terraform.tfvars` after the ALB is live): - 5xx error rate > 1% for 3 consecutive minutes → SNS `critical_alerts` topic. - p99 latency > 500ms for 3 consecutive minutes → SNS `critical_alerts` topic. - SNS `critical_alerts` topic with email subscription to `alert_email`. - Post-launch: migrate to Amazon Managed Prometheus + Grafana for richer dashboards and longer-window analysis. CloudWatch is sufficient for launch and keeps operational complexity down while the team stabilises.

SLO targets: Not yet established — no production traffic baseline exists. Working assumptions: $p99 < 500\text{ms}$ and error rate $< 0.1\%$, based on the application team's NFR document. These will be calibrated after the first real traffic window.

E. CI/CD

GitHub Actions workflow (`.github/workflows/ci-cd.yaml`) triggers on push to `main` and on pull requests.

Validate job (always runs, no AWS credentials needed): - `terraform validate` and `terraform fmt -check` (init with `-backend=false` to avoid state access). - `kustomize build` lint on the production overlay. - Grep check for unresolved placeholder tokens (`ACCOUNT_ID` , `CERT_ID` , `WAF_ID`).

Deploy job (conditional on `AWS_DEPLOY_ROLE_ARN` secret): - Authenticates to AWS via OIDC (no long-lived credentials stored in GitHub). - Builds and pushes the container image to ECR (`$ECR_REPO:$SHA`). - Substitutes account-specific values (account ID, certificate ARN, WAF ARN) into manifests via `sed` . This is pragmatic for the assessment — the production target is Terraform-generated ConfigMaps or External Secrets Operator to remove the `sed` dependency. - Verifies no placeholder tokens remain post-substitution. - Deploys via `kubectl apply` and waits for rollout status.

The deploy job is skipped by default in this assessment repository (AWS OIDC and ECR secrets not populated). Set `AWS_DEPLOY_ROLE_ARN` as a GitHub environment secret to activate live deployments.

F. Operations

Most of the Day-2 work runs itself: - Karpenter's `expireAfter: 720h` rotates nodes every 30 days, so there are no manual patching windows. - The CI pipeline validates Terraform (`validate` + `fmt -check`) and lints the Kustomize overlay on every PR. `terraform plan` is intentionally excluded from CI — it requires live AWS credentials and a bootstrapped backend, so it runs locally or in a separate privileged workflow before applying. - CloudWatch alarms are configured to fire only on business-meaningful signals (like user-facing 5xx errors or high latency) rather than noisy infrastructure metrics like high CPU on individual pods.

Team delegation (1 Senior + 2 Juniors, ~3–4 weeks):

Task	Owner	Notes
VPC design, subnet strategy, NAT gateways, VPC endpoints	Senior	Networking decisions have blast-radius implications; needs senior judgment
EKS cluster module, Karpenter v1 NodePool/EC2NodeClass, IRSA trust policies	Senior	Core platform; mistakes here cascade everywhere
KMS, WAF, Secrets Manager, security module	Senior	IAM + crypto decisions need experience
Kubernetes manifests — Deployment, Service, ConfigMap, Ingress	Junior 1	Well-scoped; senior reviews for security context
HPA, PDB, network policies	Junior 1	Pattern-driven; senior sets the acceptance criteria
Fluent Bit DaemonSet, CloudWatch log groups, SNS/alarm wiring	Junior 1	Straightforward with the IRSA role already in place
CI/CD pipeline (GitHub Actions), ECR repo, image scanning	Junior 2	Isolated from platform; good learning task
Architecture diagram and design document	All (senior owns)	Juniors contribute sections; senior writes the architecture narrative

Gating: Senior completes the VPC and EKS modules first (end of Week 1). Juniors work in parallel from Week 2 onwards on K8s manifests and CI/CD respectively. Senior reviews all PRs and unblocks on IAM questions.

Infrastructure Cost Strategy

Spot instances for burst + on-demand for baseline. Gateway endpoints (S3, DynamoDB) eliminate NAT data-processing fees on high-volume paths. Interface endpoints (ECR, STS) justified by call volume; SQS, CloudWatch Logs, and Secrets Manager route through NAT until post-launch traffic makes the cost trade-off clear.

Known Gaps

- **Application code not included** — infrastructure only, per assessment scope.
- **Canary deployments** — Currently rolling updates (`maxUnavailable: 0`); Argo Rollouts planned for request-level traffic shifting post-launch.
- **ElastiCache Redis** — Database-tier subnet scaffolding and network policy egress rule are in place; cluster provisioning deferred until the application requires it.
- **DynamoDB access patterns** — Table schema uses only `transactionId` as the hash key. Real query patterns (e.g., all transactions for a user in the last 30 days) require at least one GSI. Schema to be finalised once the application team confirms access patterns.
- **ECR lifecycle policy** — Retain last 30 images; deferred to post-launch hygiene once image cadence is known.
- **Karpenter spot interruption queue** — SQS-based interruption handling (EventBridge → SQS → Karpenter proactive drain) not provisioned; Karpenter's default 2-minute IMDS warning path handles reclamation. Add as Day-2 improvement.
- **Interface VPC endpoints for SQS, CloudWatch Logs, Secrets Manager** — not provisioned; calls route through NAT. Add post-launch.
- **SNS alarm routing** — `warning_alerts` topic exists (used as `ok_actions` on CloudWatch alarms) but has no subscription. A second email subscription would be noisy since both alarms use the same `alert_email` . Route to a separate Slack/PagerDuty endpoint or a lower-urgency email alias post-launch.
- **CloudWatch alarms require two applies** — ALB is provisioned by the Load Balancer Controller (not Terraform). Set `alb_deployed = true` after the ALB is live and re-apply.